Microsoft Teams Governance & Lifecycle Management

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary 4

Solution Overview 4

What this solution is . . . 4

How it works . . . 5

Reference design . . . 5

Benefits . . . 6

Limitations & boundaries . . . 6

Deployment model . . . 7

Requirements 7

1. Microsoft Entra ID P1 for governance features (verify plan) . . . 7

2. Microsoft Graph PowerShell (with beta directory-setting cmdlets) . . . 8

3. Directory-settings admin permissions (Group.Unified) . . . 8

4. Designated "group creators" security group . . . 8

5. Purview compliance plan for retention (verify plan) + compliance owner . . . 9

6. Container sensitivity labels published (for classification) . . . 9

7. Governance stakeholders & a naming/classification standard on paper . . . 9

Global Requirements — Microsoft Teams Solutions 10

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing . . . 10

2. Administrative access (least-privilege Teams roles) . . . 10

3. Microsoft Teams PowerShell + Graph toolchain . . . 10

4. Identity, groups & lifecycle source of truth . . . 11

5. A pilot scope and change control . . . 11

6. Security, compliance & governance stakeholders . . . 11

Implementation 12

Stage 0 — Connect . . . 12

Stage 1 — Assess (read-only) . . . 12

Stage 2 — Design . . . 12

Stage 3 — Configure (pilot scope, `-WhatIf` first) . . . 13

Stage 4 — Pilot . . . 14

Stage 5 — Validate (pass/fail gates) . . . 14

Stage 6 — Expand (production, change control) . . . 15

Testing & Validation 15

Test 1 — Creation control blocks non-creators . . . 15

Test 2 — Creation allowed for approved creators . . . 16

Test 3 — Naming policy enforced . . . 16

Test 4 — Classification label applies to a team . . . 16
Test 5 — Expiration policy active and scoped . . . 16
Test 6 — Renewal / soft-delete behavior . . . 17
Test 7 — Retention coverage (mechanism only) . . . 17
Test 8 — Ownership standard . . . 17
Test 9 — Archival read-only . . . 17
Test 10 — Governance report accuracy . . . 18
Troubleshooting 18
Creation control . . . 18
Naming policy . . . 19
Classification (container labels) . . . 19
Expiration . . . 19
Retention . . . 20
Ownership . . . 20
Archival . . . 20
Reporting & validation . . . 21
Best Practices 21
Governance model . . . 21
Naming & classification . . . 21
Lifecycle: expiration, retention, archival . . . 22
Ownership . . . 22
Operations & safety . . . 22
Reporting & continuous improvement . . . 23
Deliverables Summary 23
Documentation . . . 23
PowerShell scripts (5) . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary

Outcome. Microsoft Teams stops sprawling and comes under managed governance. Team/M365 Group creation is restricted to approved creators, every team is named and classified to a standard, ownership is enforced, and inactive teams are expired, retained, or archived on a defined lifecycle — with a repeatable report that proves the posture on demand.

The problem it solves. Left open, Teams grows uncontrolled: anyone creates a team, duplicates multiply, owners depart and are never replaced, and data ages indefinitely with no expiration or retention decision behind it. The result is duplicate and orphaned teams, unclassified sensitive content, and an estate no one can account for.

Approach. The solution establishes five native controls — **creation control**, **naming & classification**, **membership/ownership standards**, **expiration + retention + archival**, and **governance reporting** — all driven by PowerShell (Microsoft Graph, MicrosoftTeams, and Purview) with no hardcoded secrets. It ships run-to-implement documentation plus five scripts: discovery/gap assessment, creation+naming configuration, lifecycle (expiration/retention/archival) configuration, a governance report, and a validation gate.

Why it is low-risk. Every change follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Turning off open creation and enabling tenant-wide expiration are high-impact, so every write script defaults to a `-WhatIf` **dry-run**, changes are applied **pilot-first** (expiration scoped to selected groups before `All`), each stage has explicit **PASS/FAIL gates**, and a **documented rollback** (re-enable creation, remove expiration, un-archive) is ready throughout. Licensing- and cloud-dependent behavior (Entra ID P1, Purview plans, Government-cloud URLs) is flagged to verify rather than assumed, and all **compliance determinations** — retention durations, records schedules, legal holds — are deferred to the organization's compliance/records/legal functions, which own the decision while this solution provides the technical method and evidence.

Bottom line. Teams that scale without chaos: creation governed, teams named and classified, ownership enforced, and stale teams expired or archived — collaboration under control, with the evidence to prove it.

Solution Overview

What this solution is

Microsoft Teams Governance & Lifecycle Management is a run-to-implement solution that puts an enterprise back in control of how Microsoft Teams is created, named, classified, owned, and retired. Every team is backed by a **Microsoft 365 Group**, so "Teams governance" is really governance of the underlying M365 Groups plus the Teams-specific surfaces (channels, archival, reporting). Without governance, Teams grows into **sprawl**: anyone can spin up a team, duplicates multiply, owners leave and are never replaced, and data ages indefinitely with no expiration or retention decision behind it.

This solution establishes five governance controls and the lifecycle behind them:

1. **Creation control** — restrict who can create teams/M365 Groups to an approved **group-of-creators**

(or a request/approval model), instead of letting every user create at will.

1. **Naming & classification policy** — a tenant naming standard (prefix/suffix + blocked words) and a

classification decision on every team (via container **sensitivity labels**), so teams are self-describing and their sensitivity is visible.

1. **Membership & ownership standards** — a minimum-owners rule and an ownerless-remediation path so no

team is left without an accountable owner.

1. **Expiration, retention & archival** — an **M365 Groups expiration policy** so inactive teams are

renewed-or-removed, **Microsoft Purview retention** for Teams content (the *schedule* is the compliance team's determination — you deliver the technical method), and **archival** of teams that should be kept read-only rather than deleted.

1. **Governance reporting** — a repeatable report of creation-control status, naming/classification

coverage, ownerless/stale/duplicate teams, and expiration posture.

How it works

The controls live in three planes, all driven by **PowerShell** (no hardcoded secrets):

• **Directory settings (`Group.Unified` template)** — the tenant-wide behavior for M365 Group creation,

the group naming policy (prefix/suffix + custom blocked words), and the switch that turns on sensitivity labels for groups. Managed with the **Microsoft Graph PowerShell (beta) directory-setting cmdlets** (`Get-MgBetaDirectorySetting` / `Update-MgBetaDirectorySetting`).

• **Group lifecycle policy** — the **expiration** policy (lifetime in days, which groups it covers,

renewal notifications) managed with `*-MgGroupLifecyclePolicy`.

• **Purview & Teams surfaces** — **retention policies** for Teams channel messages / chats and the

associated SharePoint/OneDrive files (Security & Compliance PowerShell), **container sensitivity labels** for classification (Purview), and **team archival** with `Set-TeamArchivedState` (MicrosoftTeams module).

> ***Plan / licensing note (verify in your tenant).*** *The M365 Groups **naming policy**, **expiration policy**, and **container sensitivity labels for groups** require **Microsoft Entra ID P1** for the affected users, and **Purview retention** requires the appropriate compliance plan. Commercial vs Government (GCC/GCC High/DoD) tenants use different portal URLs (e.g. `*.office365.us`) and some features roll out differently — confirm availability and the exact portal path in your own tenant.*

Reference design

A worked reference used throughout the implementation, testing, and scripts. Reuse these exact artifact names when you implement:

Artifact	Name (convention `<Purpose> – <Scope> – <Stage>`)	Purpose
Creators security group (pilot)	`Group Creators – Governed – Pilot`	Members who may create M365 Groups/Teams during pilot

Artifact	Name (convention <Purpose> – <Scope> – <Stage>)	Purpose
Creators security group (prod)	`Group Creators – Governed – Prod`	Approved creators tenant-wide after expansion
Naming policy	`Naming Policy – M365 Groups – Prod`	Prefix/suffix + blocked-words standard
Classification labels	`Sensitivity Label – Public/Internal/Confidential – Prod`	Container labels applied to teams
Expiration policy	`Expiration Policy – M365 Groups – Prod`	Renew-or-remove lifetime for teams
Retention policy	`Retention Policy – Teams – Prod`	Purview retention for Teams messages + files (schedule set by compliance)
Ownerless remediation	`Ownerless Policy – Teams – Prod`	Prompts members to become owners
Pilot team set	`Pilot Team – Governance-A – Pilot` …	The teams the pilot validates against

Benefits

• **Sprawl stops at creation** — only approved creators (or an approved request) can make a team, so duplicates and shadow teams stop multiplying.

• **Every team is self-describing** — naming standard + classification make ownership, purpose, and sensitivity visible at a glance and queryable in reporting.

• **No orphaned data** — expiration renews-or-removes inactive teams, retention satisfies the compliance schedule, and archival keeps must-retain teams read-only instead of deleting them.

• **Accountable ownership** — minimum-owners and ownerless remediation keep every team owned.

• **Evidence, not guesswork** — the reporting script produces a defensible governance posture on demand.

Limitations & boundaries

• **Compliance/records/legal determinations are deferred.** Retention **durations**, records schedules, legal-hold scope, and which classification maps to which policy are decisions for the org's compliance/records/legal functions. This solution delivers the **technical method and evidence**, not the determination.

• **Licensing-gated features are flagged, not assumed.** Naming policy, expiration, container labels, and

retention depend on Entra ID P1 / Purview plans — verify in your tenant.

• **A full self-service request portal** (approval workflow beyond the creators-group model) is an

optional extension (e.g. a Power Automate / ticketing front end) referenced at the pattern level.

• **Third-party governance/provisioning tools** are out of scope; this solution uses native M365 controls.

Deployment model

This solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — because turning off open group creation and enabling expiration are **high-impact, tenant-wide** changes:

1. **Assess** — inventory existing teams/groups; find ownerless, stale, and duplicate teams (read-only).

2. **Design** — decide the creators-group model, naming standard, classification scheme, expiration

lifetime, retention posture (with compliance), and archival criteria.

1. **Configure** — apply each control **against the pilot scope** first, using `-WhatIf` dry-runs.

2. **Pilot** — validate on `Group Creators – Governed – Pilot` and a small pilot team set.

3. **Validate** — run the validation gates (creation controlled, naming enforced, expiration active,

stale teams handled); confirm each **pass/fail** gate.

1. **Expand** — roll to production (… `– Prod`) under change control, with a **documented rollback**

(re-enable creation, remove naming/expiration) ready at every step.

Requirements

These are the **solution-specific** prerequisites for Teams governance & lifecycle. They build **on top of** the shared **Global Requirements — Microsoft Teams Solutions** (`global-requirements-teams.md`): a licensed M365 tenant with Teams (GR-1), least-privilege admin roles (GR-2), the MicrosoftTeams + Graph toolchain (GR-3), an identity/group/lifecycle source of truth (GR-4), a pilot scope + change control (GR-5), and security/compliance/governance stakeholders (GR-6). Verify the shared six first, then the items below. Each item: **Description / Why / Verify / Implement if missing**.

1. Microsoft Entra ID P1 for governance features (verify plan)

• **Description:** Microsoft Entra ID **P1** licensing covering every user who is a member of a governed

group, which gates the M365 Groups **naming policy**, **expiration policy**, and **group sensitivity (container) labels**.

• **Why:** These three core controls will not apply — or will error — without P1 for the affected users.

This is the most common blocker for a governance rollout.

• **Verify:** In the Microsoft 365 admin center → **Billing** → **Licenses**, confirm Entra ID P1 (or a suite that includes it, e.g. certain E-plans) is assigned to governed users. Flag Commercial vs Government — confirm availability in your cloud.

• **Implement if missing:** Acquire/assign Entra ID P1 to the users in scope before enabling naming, expiration, or container labels.

2. Microsoft Graph PowerShell (with beta directory-setting cmdlets)

• **Description:** The **Microsoft.Graph** and **Microsoft.Graph.Beta** PowerShell modules, in addition to the MicrosoftTeams module from GR-3. The `Group.Unified` directory settings (creation control, naming policy) are managed with the **beta** directory-setting cmdlets.

• **Why:** Creation control and the naming policy are stored as directory settings, exposed today through `Get-MgBetaDirectorySetting` / `Update-MgBetaDirectorySetting`. The lifecycle (expiration) policy uses `*-MgGroupLifecyclePolicy`.

• **Verify:** `Get-Module Microsoft.Graph.Beta -ListAvailable` returns a version; `Connect-MgGraph -Scopes "Directory.ReadWrite.All","Group.ReadWrite.All"` succeeds.

• **Implement if missing:** `Install-Module Microsoft.Graph`, `Install-Module Microsoft.Graph.Beta`; confirm modern-auth/MFA sign-in. (The legacy AzureAD/AzureADPreview modules are deprecated — use Graph.)

3. Directory-settings admin permissions (Group.Unified)

• **Description:** An admin identity able to read and write **directory settings** and group lifecycle — practically a role with the `Directory.ReadWrite.All` and `Group.ReadWrite.All` Graph scopes (e.g. **Groups Administrator** for lifecycle/naming, with **Global Administrator** only where a setting genuinely requires it).

• **Why:** Changing the `Group.Unified` template (creation control, naming) and creating a lifecycle policy are directory-level writes.

• **Verify:** `Get-MgBetaDirectorySetting` lists the tenant settings without an authorization error.

• **Implement if missing:** Have a Global Admin grant the least-privilege role/consent the required Graph scopes under change control.

4. Designated "group creators" security group

• **Description:** A **security group** (mail-enabled not required) whose members are the users permitted to create M365 Groups/Teams once open creation is turned off — named `Group Creators – Governed – Pilot` for the pilot and `Group Creators – Governed – Prod` for production.

• **Why:** Creation control works by setting `EnableGroupCreation = false` and pointing

`GroupCreationAllowedGroupId` at exactly one security group. You must have that group first.

• **Verify:** The group exists in Entra ID and its `Id` (GUID) is known.

• **Implement if missing:** Create the security group and enroll the approved creators (service desk, governance team, or delegated owners) before applying creation control.

5. Purview compliance plan for retention (verify plan) + compliance owner

• **Description:** The **Microsoft Purview** licensing that enables **retention policies** for Teams (channel messages, chats) and the files in the backing SharePoint/OneDrive, plus a named **compliance/ records owner** who sets the retention **schedule**.

• **Why:** Retention durations, records schedules, and legal-hold scope are **compliance determinations**, not technical defaults — this solution configures the mechanism, the owner sets the policy.

• **Verify:** `Connect-IPPSSession` (Security & Compliance PowerShell) connects; `Get-RetentionCompliancePolicy` runs. Confirm the plan covers Teams locations. Flag Commercial vs Government portal/URL differences.

• **Implement if missing:** Assign the compliance plan; identify the retention owner and capture the schedule before creating any retention policy.

6. Container sensitivity labels published (for classification)

• **Description: Microsoft Purview** container sensitivity labels (e.g. Public / Internal / Confidential) that are scoped to **Groups & sites** and **published** to the users who create teams, used as the classification mechanism.

• **Why:** Modern Teams/Groups classification uses **container labels** (the legacy `ClassificationList` is deprecated). Enabling `EnableMIPLabels = true` requires labels that already exist and are published.

• **Verify:** In Purview → **Information protection** → **Labels**, confirm labels scoped to "Groups & sites" exist and a label policy publishes them; `Get-Label` (Security & Compliance PowerShell) lists them.

• **Implement if missing:** With the compliance owner, create and publish container labels **before** turning on `EnableMIPLabels`.

7. Governance stakeholders & a naming/classification standard on paper

• **Description:** An agreed **naming standard** (prefix/suffix pattern + blocked words), **classification scheme **(which label for which data),** minimum-owners **rule,** expiration lifetime**, and** archival criteria** — signed off by the collaboration-governance stakeholders (extends GR-6).

• **Why:** The scripts and steps encode these decisions; configuring before the standard is agreed causes churn and user-facing disruption.

• **Verify:** A short written standard exists and names an approver.

• **Implement if missing:** Run a design workshop with governance/security/compliance to agree the standard before the Configure stage.

Global Requirements — Microsoft Teams Solutions

These requirements apply to **every** solution in the **Teams** category (Teams governance, meetings, external access, Teams Phone, apps, Teams Rooms, and Teams security/compliance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Teams solution).*** *The **Teams admin center** (admin.teams.microsoft.com), the **Microsoft 365 admin center**, the **MicrosoftTeams PowerShell module**, and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Where a capability, policy, or portal path is **plan- or version-dependent** (Teams Phone licensing, Calling Plans vs Operator Connect vs Direct Routing, Premium features, Commercial vs Government cloud), it is **cited or flagged "verify in your tenant"** — never invented. **Third-party devices and apps** (certified Teams Rooms hardware, SBCs, ISV apps) are integrated at the pattern level; their internals are deferred to the vendor's own docs. Automation is **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing

• **Description:** A Microsoft 365 tenant with **Microsoft Teams** enabled and the licensing the capability requires (e.g. Teams Phone + a calling plan/Operator Connect for voice, Teams Premium for premium features, the appropriate E-plan for advanced compliance).

• **Why required:** All policy, voice, app, and compliance configuration depends on the right plan.

• **How to verify:** In the M365 admin center confirm the Teams subscription and relevant add-ons.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Administrative access (least-privilege Teams roles)

• **Description:** Access with the least-privilege role the solution names — typically **Teams Administrator **(or Teams Communications/Device Administrator), with** Global Administrator** only where genuinely required.

• **Why required:** Teams policy, voice, app, and org-wide settings require the corresponding role.

• **How to verify:** Confirm the role in the M365 admin center / Entra roles.

• **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

3. Microsoft Teams PowerShell + Graph toolchain

• **Description:** A workstation with the **MicrosoftTeams** PowerShell module (and Graph PowerShell

where used), modern-auth / MFA-capable admin sign-in.

- **Why required:** Configuration, automation, and the provided scripts run through Teams PowerShell / Graph.
- **How to verify:** `Connect-MicrosoftTeams` succeeds; `Get-Module MicrosoftTeams`.
- **How to implement if missing:** `Install-Module MicrosoftTeams`; confirm modern-auth sign-in.

4. Identity, groups & lifecycle source of truth

- **Description:** A defined identity/group source (Entra ID groups, and where used **Microsoft 365 Groups **which back teams) and the** lifecycle** stance (creation, naming, expiration/retention) the org expects.
- **Why required:** Teams are backed by M365 Groups; governance, provisioning, and access key off groups and the lifecycle model.
- **How to verify:** Confirm which groups/OU model is authoritative and the lifecycle expectations.
- **How to implement if missing:** Define the group/lifecycle model before configuring governance.

5. A pilot scope and change control

- **Description:** A **pilot team / policy group / user set** to validate on before org-wide rollout, and a **change-control** process for production policy/voice/compliance changes.
- **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model; policy and voice changes are high-impact.
- **How to verify:** Confirm a named pilot scope and an approval/change window are available.
- **How to implement if missing:** Create a pilot scope and agree change control before production changes.

6. Security, compliance & governance stakeholders

- **Description:** Engagement with the **security, compliance/records, and collaboration-governance** stakeholders whose policies the configuration must satisfy (external access, information barriers, retention, DLP, app governance).
- **Why required:** Collaboration governance and compliance settings must align to enterprise policy.
- **How to verify:** Confirm the applicable policies and named approvers.
- **How to implement if missing:** Identify the owners and capture requirements before configuring governance.

*The deployment model across every Teams solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets in code.*

Implementation

Follow the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Turning off open group creation and enabling expiration are **tenant-wide, high-impact** changes, so every Configure step is done **pilot-first**, with a `-WhatIf` **dry-run** before any commit and a **rollback** noted inline. Reuse the exact artifact names from the reference design.

> ***Naming convention (mandatory).*** *Every artifact is named* `<Purpose> – <Scope> – <Stage>` *and reused verbatim in testing, troubleshooting, and the scripts — e.g.* `Group Creators – Governed – Pilot`, `Expiration Policy – M365 Groups – Prod`.

Stage 0 — Connect

```
Connect-MgGraph -Scopes "Directory.ReadWrite.All","Group.ReadWrite.All","GroupMember.ReadWrite.All"
Connect-MicrosoftTeams
Connect-IPPSSession        # Security & Compliance PowerShell (retention, labels)
```

Use a modern-auth/MFA admin sign-in. **No stored credentials.** For Government clouds pass the environment switches your tenant requires (e.g. `Connect-MgGraph -Environment USGov`) — verify the exact value in your tenant.

Stage 1 — Assess (read-only)

1. **Inventory every team/M365 Group.** Run `Get-TeamsGovernanceInventory.ps1` (Script 1). It enumerates

all M365 Groups/teams and flags **ownerless**, **stale** (no recent activity), potential **duplicates** (near-identical display names), and **unclassified** teams. Nothing is changed.

1. **Capture the baseline.** Save the inventory CSV as the pre-change baseline; it is your rollback

reference and the "before" side of the governance report.

1. **Confirm current creation posture.** Note whether `EnableGroupCreation` is currently unset/`true`

(open creation) — the inventory report surfaces this.

Gate 1 (Assess): You have a complete inventory and a written list of ownerless/stale/duplicate teams.

Stage 2 — Design

1. **Creators model.** Confirm the security group `Group Creators – Governed – Pilot` (and later

… `– Prod`) and its membership.

1. **Naming standard.** Decide the prefix/suffix and blocked words for `Naming Policy – M365 Groups – Prod`

— e.g. prefix `GRP-[Department]-` , suffix `-[GroupName]`, blocked words like `CEO`, `Payroll`, `HR`.

1. **Classification.** Confirm the published container labels `Sensitivity Label – Public/Internal/

Confidential – Prod` and which data maps to which (compliance owner decides).

1. **Lifecycle.** Decide the expiration lifetime (e.g. 180 or 365 days) for `Expiration Policy – M365

Groups – Prod, the retention posture for Retention Policy – Teams – Prod` (**duration set by compliance**), and archival criteria (e.g. stale > 180 days AND owner confirms keep-read-only).

1. **Ownership.** Set the minimum-owners rule (e.g. ≥ 2 owners) and the ownerless remediation approach.

Gate 2 (Design): A one-page standard is signed off by governance/security/compliance.

Stage 3 — Configure (pilot scope, `-WhatIf` first)

3a. Creation control + naming policy

1. **Dry-run.** `.\Set-TeamsCreationNamingPolicy.ps1 -CreatorsGroupName "Group Creators – Governed – Pilot"

-Prefix "GRP-" -BlockedWords "CEO,Payroll,HR" -WhatIf – shows the exact Group.Unified settings that would change (EnableGroupCreation, GroupCreationAllowedGroupId, PrefixSuffixNamingRequirement, CustomBlockedWordsList`) without writing.

1. **Commit (pilot).** Re-run with `-Commit`. This sets `EnableGroupCreation = false` and points

`GroupCreationAllowedGroupId` at `Group Creators – Governed – Pilot`, and applies the naming policy.

> ***Note:*** *creation control is **tenant-wide** — there is no per-scope creation setting. Pilot the *naming policy string* and the *creators membership* by keeping the creators group small first; only expand membership at Stage 6. Announce the change to users before committing.*

1. **Rollback:** re-run with `-Commit -Rollback` to set `EnableGroupCreation = true` and clear the naming

requirement, restoring open creation.

3b. Classification (container labels)

1. Ensure `Sensitivity Label – … – Prod` labels are **published** (Requirement 6).

2. Enable label support: the same config script sets `EnableMIPLabels = true` when `-EnableLabels` is

passed (dry-run first). After enabling, run the Graph label sync as documented (`Execute-AzureAdLabelSync` equivalent via Graph) — **verify the current sync cmdlet in your tenant**, as this step is version-dependent.

1. Apply a label to a pilot team to confirm the label appears on the team/site.

3c. Ownership standards

1. Configure the **ownerless groups policy** `Ownerless Policy – Teams – Prod` in the Microsoft 365 admin

center → **Settings** → **Org settings** → **Microsoft 365 Groups** → **Ownerless groups**, selecting members to be prompted to become owners. (This is an admin-center feature; confirm availability in your cloud.)

1. For the pilot, use `Get-TeamsGovernanceInventory.ps1` output to identify pilot teams lacking the

minimum owners and assign owners with `Add-MgGroupOwnerByRef` (or via the admin center).

3d. Expiration + retention + archival

1. **Expiration dry-run.** `.\Set-TeamsLifecyclePolicy.ps1 -ExpirationDays 180 -ManagedGroupTypes Selected

-PilotGroupId <pilotGroupGuid> -NotificationEmails "governance@contoso.com" -WhatIf`. Using -ManagedGroupTypes Selected` scopes expiration to the pilot groups only — **this is how you pilot expiration safely**.

1. **Commit (pilot).** Re-run with `-Commit` to create `Expiration Policy – M365 Groups – Prod` covering

only the selected pilot groups. Owners receive renewal notifications; unrenewed groups are soft-deleted (restorable for 30 days by default — verify).

1. **Retention.** With the compliance owner, create `Retention Policy – Teams – Prod` for Teams channel

messages/chats and the associated files. The lifecycle script will **report** the current retention coverage and, when `-ConfigureRetention -Commit` is passed with an explicit `-RetentionDays` supplied **by compliance**, create the Purview policy via `New-RetentionCompliancePolicy` / `New-RetentionComplianceRule`. **Never** set a retention duration without the compliance determination.

1. **Archival.** For teams that should be kept read-only rather than deleted, the lifecycle script archives

them with `Set-TeamArchivedState -GroupId <id> -Archived $true -SetSpoSiteReadOnlyForMembers $true` (dry-run first; `-Commit` to apply). Rollback: `Set-TeamArchivedState … -Archived $false`.

Gate 3 (Configure): Every control applied to the pilot with a successful dry-run first; rollback tested for creation control and archival.

Stage 4 — Pilot

1. Have a pilot user **not** in `Group Creators – Governed – Pilot` attempt to create a team — it must be

blocked.

1. Have a creator create a team with a non-compliant name — the naming policy must block/prefix it.
2. Apply/observe a classification label on `Pilot Team – Governance-A – Pilot`.
3. Let a selected pilot group approach expiration (or force a short lifetime in a test) and confirm the

renewal notification and soft-delete/renew behavior.

1. Archive one stale pilot team and confirm read-only.

Stage 5 — Validate (pass/fail gates)

Run `Test-TeamsGovernanceReadiness.ps1` (Script 5) and confirm each gate:

- **Creation controlled** — `EnableGroupCreation = false`, `GroupCreationAllowedGroupId` set. **PASS/FAIL**
- **Naming enforced** — `PrefixSuffixNamingRequirement` present; a bad name is rejected. **PASS/FAIL**
- **Classification on** — `EnableMIPLabels = true`; pilot team shows a label. **PASS/FAIL**
- **Expiration active** — a lifecycle policy exists with the intended lifetime. **PASS/FAIL**
- **Ownership** — pilot teams meet the minimum-owners rule; ownerless policy configured. **PASS/FAIL**
- **Stale handled** — stale pilot teams are archived or slated for expiration. **PASS/FAIL**

Gate 5 (Validate): All gates PASS on the pilot before expanding.

Stage 6 — Expand (production, change control)

1. Repoint creation control from the pilot creators group to `Group Creators – Governed – Prod` (re-run

Script 2 with the prod group; dry-run first).

1. Change expiration from `-ManagedGroupTypes Selected` to `All` (re-run Script 3 with `-ManagedGroupTypes

All`) so **every** group is covered — announce first; this soft-deletes unrenewed groups tenant-wide.

1. Publish the naming/classification standard to all users; expand creators-group membership per the model.
2. Schedule `Get-TeamsGovernanceReport.ps1` (Script 4) to run on a cadence (e.g. monthly) for ongoing

posture and to catch drift.

Documented rollback (any stage): re-enable open creation (`Set-TeamsCreationNamingPolicy.ps1 -Commit -Rollback`), remove/relax expiration (`Set-TeamsLifecyclePolicy.ps1 -RemoveExpiration -Commit`), and un-archive teams (`Set-TeamArchivedState … -Archived $false`). Keep the Stage-1 baseline inventory as the reference for what "before" looked like.

Testing & Validation

Each test states the **procedure** and the **expected pass/fail** signal. Run them after the pilot configure stage (Stage 3) and again after production expansion (Stage 6). Most map directly to a check in `Test-TeamsGovernanceReadiness.ps1` (Script 5).

Test 1 — Creation control blocks non-creators

• **Procedure:** As a user who is **not** a member of `Group Creators – Governed – Pilot`, attempt to

create a team (Teams client → *Join or create a team*) and an M365 Group (e.g. a new Planner/SharePoint group). Also run `Get-MgBetaDirectorySetting` and inspect the `Group.Unified` values.

• **Pass:** Creation is blocked/hidden for the non-creator; `EnableGroupCreation = false` and

`GroupCreationAllowedGroupId` equals the creators group's GUID.

• **Fail:** The non-creator can still create a team, or the settings are unset.

Test 2 — Creation allowed for approved creators

• **Procedure:** As a member of `Group Creators – Governed – Pilot`, create a team.

• **Pass:** The team is created successfully.

• **Fail:** An approved creator is blocked (check group membership propagation — allow time for replication).

Test 3 — Naming policy enforced

• **Procedure:** As an approved creator, create a group named to violate the standard — include a blocked

word (e.g. `Payroll`) and omit the required prefix.

• **Pass:** The blocked word is rejected, and the configured prefix/suffix

(`PrefixSuffixNamingRequirement`) is auto-applied to a valid name.

• **Fail:** The blocked word is accepted or no prefix/suffix is applied.

Test 4 — Classification label applies to a team

• **Procedure:** Ensure `EnableMIPLabels = true`. Create/label `Pilot Team – Governance-A – Pilot` with

`Sensitivity Label – Confidential – Prod`. Check the team/site shows the label.

• **Pass:** The container label is visible on the team and its SharePoint site; label-driven settings

(guest access/privacy) apply per the label.

• **Fail:** No label option appears, or the label does not stick (confirm the label is published and scoped

to Groups & sites, and the label sync has run — version-dependent, verify in your tenant).

Test 5 — Expiration policy active and scoped

• **Procedure:** Run `Get-MgGroupLifecyclePolicy`. For the pilot, confirm `managedGroupTypes = Selected`

and the pilot group is included; check `groupLifetimeInDays` matches the design.

• **Pass:** A single lifecycle policy exists with the intended lifetime and scope; renewal notifications go to `alternateNotificationEmails`.

• **Fail:** No policy, wrong lifetime, or `All` when a scoped pilot was intended.

Test 6 — Renewal / soft-delete behavior

• **Procedure:** In a controlled test, let a selected group reach expiration (or use a short test lifetime). Observe owner renewal notifications; do not renew; confirm soft-delete, then restore.

• **Pass:** Owners are notified; an unrenewed group is soft-deleted and is **restorable** within the deleted-groups window (default ~30 days — verify); a renewed group persists.

• **Fail:** No notification, or the group is hard-deleted / not restorable.

Test 7 — Retention coverage (mechanism only)

• **Procedure:** Run `Get-RetentionCompliancePolicy` / `Get-RetentionComplianceRule`; confirm `Retention Policy – Teams – Prod` targets the Teams channel/chat locations and the associated files, with the **duration supplied by compliance**.

• **Pass:** The policy exists, is enabled, and covers the intended Teams locations. (The **correctness of the duration** is the compliance owner's determination, not this test.)

• **Fail:** No policy, disabled, or missing the Teams locations.

Test 8 — Ownership standard

• **Procedure:** Run `Get-TeamsGovernanceInventory.ps1` and filter to teams below the minimum-owners rule. Confirm the ownerless groups policy `Ownerless Policy – Teams – Prod` is configured in Org settings.

• **Pass:** Pilot teams meet the minimum-owners threshold; the ownerless policy is enabled and prompts members.

• **Fail:** Ownerless/under-owned pilot teams remain, or the policy is not configured.

Test 9 — Archival read-only

• **Procedure:** Archive a stale pilot team via the lifecycle script (`-Commit`). Confirm with `Get-Team | Where-Object Archived` and try to post in a channel as a member.

• **Pass:** The team shows `Archived = true`; members cannot post (SharePoint site read-only when `-SetSpoSiteReadOnlyForMembers` was used). Un-archive restores write access.

• **Fail:** The team is not archived or members can still post.

Test 10 — Governance report accuracy

- **Procedure:** Run `Get-TeamsGovernanceReport.ps1`. Cross-check a sample of its counts (creation status,

naming, classification coverage, ownerless/stale/duplicate, expiration posture) against the admin center / raw cmdlet output.

- **Pass:** Report figures reconcile with the source data; the report renders (CSV/HTML) without error.
- **Fail:** Counts diverge from source, or the report errors (check connection scopes and permissions).

Overall pass/fail gate

The solution is validated when **Tests 1–6, 8, and 9 PASS on the pilot**, **Test 7** confirms the retention **mechanism** is in place (duration owned by compliance), and **Test 10** reconciles. Only then proceed to Stage 6 (Expand). Re-run Tests 1, 3, 5, and 10 after production expansion.

Troubleshooting

Common failure modes and resolutions. Symptoms map to the controls in the implementation and the five scripts. For plan/cloud-dependent behavior, **verify in your tenant** rather than assuming.

Creation control

Users can still create teams after applying creation control.

- **Directory changes have not propagated.** `Group.Unified` changes can take time to reach all users;

wait and re-test. Confirm the write with `Get-MgBetaDirectorySetting`.

- `EnableGroupCreation` **still** `true`**.** Re-run `Set-TeamsCreationNamingPolicy.ps1` with `-Commit` and

confirm the re-read shows `false`. A dry-run (missing `-Commit`) never writes.

- **Wrong or empty** `GroupCreationAllowedGroupId`**.** Confirm it holds the creators group **GUID**, not the

name. Run `Test-TeamsGovernanceReadiness.ps1 -CreatorsGroupName "…"` to check the binding.

Approved creators are blocked.

- **Membership not yet replicated** in the creators security group — allow time and re-test.
- **Nested groups aren't honored** — `GroupCreationAllowedGroupId` uses direct membership of one security

group; add users directly (or the intended nested group's members).

`Get-MgBetaDirectorySetting` **errors or returns nothing.**

- **Beta module missing** — `Install-Module Microsoft.Graph.Beta`.
- **Insufficient scope** — reconnect with `Connect-MgGraph -Scopes "Directory.ReadWrite.All"`.
- **Setting not yet created** — the tenant may have no `Group.Unified` instance; the config script creates

it from the template on first `-Commit`.

Naming policy

Blocked words / prefix not enforced.

- **Entra ID P1 not licensed** for the creating user — the naming policy is P1-gated; assign P1 (verify).
- **Policy not written** — confirm `PrefixSuffixNamingRequirement` / `CustomBlockedWordsList` are set via

`Get-MgBetaDirectorySetting`.

- **Admin bypass** — global/groups admins can bypass naming policy by design; test with a standard creator.

Names get an unexpected prefix/suffix.

- That is the policy working. Review the `PrefixSuffixNamingRequirement` string (e.g. `GRP-[GroupName]`)

and adjust with the config script if the pattern is wrong.

Classification (container labels)

No label option when creating a team, or labels don't stick.

- `EnableMIPLabels` **not** `true` — enable it (config script `-EnableLabels -Commit`).
- **Labels not published or wrong scope** — the container labels must exist, be scoped to **Groups &

sites**, and be** published** to the creator (Requirement 6); confirm with `Get-Label`.

- **Label sync not run** — after enabling label support a directory label sync is required; the exact

cmdlet is **version-dependent — verify the current sync step in your tenant**.

- **P1 not licensed** — container labels for groups require Entra ID P1.

Expiration

No renewal notifications / groups not expiring.

- **P1 not licensed** for members — expiration requires Entra ID P1 for all group members (verify).
- **Scope is** `Selected` **but group not enrolled** — enroll the group (`Add-MgGroupToLifecyclePolicy`) or

switch to `All` for tenant-wide coverage.

- **Notification target wrong** — for groups without an owner, notifications go to

`alternateNotificationEmails`; ensure it points at a monitored governance mailbox.

- **Auto-renew on activity** — active groups auto-renew and will not expire; this is expected.

A group was deleted by expiration and users want it back.

- **Restore from soft-delete** — an expired group is soft-deleted and restorable within the deleted-groups

window (default ~30 days — **verify**). Restore it, then investigate why it wasn't renewed.

`New-MgGroupLifecyclePolicy` **fails "policy already exists".**

- **Only one lifecycle policy per tenant is allowed.** Update the existing one (`Update-MgGroupLifecyclePolicy`)

— the config script does this automatically when a policy is present.

Retention

No retention policy created despite `-ConfigureRetention -Commit`**.**

- **By design:** the script refuses to create retention without an explicit `-RetentionDays` **supplied by

compliance**. Obtain the duration from the compliance/records owner and pass it.

- **Not connected to Security & Compliance** — run `Connect-IPPSSession` first.

- **Plan/portal differences** — Purview retention coverage and portal URLs differ between Commercial and

Government clouds (`*.office365.us`); verify the plan covers the Teams locations.

Expired group's content unexpectedly retained.

- Expiration and retention are independent — content under `Retention Policy – Teams – Prod` is preserved

for the compliance duration even after the group is expired. This is correct behavior.

Ownership

Ownerless teams keep appearing.

- **Ownerless groups policy not configured** — set `Ownerless Policy – Teams – Prod` in Org settings so

members are prompted to become owners.

- **Owners leaving** — schedule `Get-TeamsGovernanceInventory.ps1` monthly to resurface under-owned teams;

enforce the ≥ 2 owners rule.

Archival

`Set-TeamArchivedState` **fails or members can still post.**

- **Not connected** — run `Connect-MicrosoftTeams`.

- **Site not read-only** — pass `-SetSpoSiteReadOnlyForMembers $true` when archiving; the lifecycle script

does this by default.

- **Wrong identifier** — use the team's **GroupId** (GUID), not display name.

Reporting & validation

Report counts don't match the admin center.

- **Timing / caching** — Graph activity and admin-center reports can lag; re-run and compare like-for-like.
- **Scope gaps** — ensure the connection has `Group.Read.All`, `Directory.Read.All`, and

`GroupMember.Read.All`.

`Test-TeamsGovernanceReadiness.ps1` **exits non-zero.**

- That is the gate working — read the FAIL lines. Resolve each failed control (creation, naming,

classification, expiration, ownership, stale) before running Stage 6 (Expand).

Best Practices

Durable practices for running Teams governance & lifecycle over time.

Governance model

- **Govern at creation, not after.** The cheapest sprawl to remove is the team that was never created.

Keep `EnableGroupCreation = false` and manage a small, well-known `Group Creators – Governed – Prod` group (or a request/approval front end) rather than reactively cleaning up.

- **One creators group, clearly owned.** `GroupCreationAllowedGroupId` accepts exactly one security group.

Keep its membership deliberate and reviewed; treat additions as a governance decision.

- **Decisions on paper before settings in the tenant.** The naming standard, classification scheme,

expiration lifetime, and archival criteria should be signed off before you touch directory settings — configuring first causes user-facing churn.

Naming & classification

- **Make names self-describing and queryable.** A consistent prefix/suffix (department, function) lets the

reporting script and admins slice the estate. Reserve blocked words for the terms that must never appear (`CEO`, `Payroll`, executive names).

- **Use container sensitivity labels, not legacy classifications.** The old `ClassificationList` is

deprecated; standardize on published container labels scoped to Groups & sites, and let the label drive privacy/guest settings so classification and access stay coupled.

- **Keep the label set small.** Three to five container labels are easier to apply correctly than a long

list; correctness beats granularity.

Lifecycle: expiration, retention, archival

- **Separate expiration from retention.** Expiration removes *inactive* groups; retention *preserves*

content for a compliance duration. They serve different owners and must not be conflated — a group can be expired while its content is still under a retention hold.

- **Retention duration belongs to compliance.** Never set a retention or records duration from a technical

default; capture it from the compliance/records owner and record who decided it.

- **Prefer archive over delete for ambiguous teams.** When in doubt, `Set-TeamArchivedState` keeps the team

read-only and reversible instead of a hard delete; expiration soft-delete is restorable only within the deleted-groups window (verify duration in your tenant).

- **Renewal notifications go to a monitored mailbox.** Point `alternateNotificationEmails` at a governance

distribution list, not one person, so renewals aren't missed when someone is out.

- **Auto-renew on activity.** M365 Groups expiration auto-renews groups that show activity — rely on it so

active teams are not falsely expired, and target the manual review at genuinely idle groups.

Ownership

- **Enforce a minimum of two owners.** A single owner is a single point of failure; require ≥ 2 and use the

ownerless groups policy to recruit owners automatically before a team goes orphaned.

- **Re-check ownership on a cadence.** Owners leave the org; schedule the inventory script to resurface

ownerless/under-owned teams monthly.

Operations & safety

- **Always dry-run first.** Every write script defaults to `-WhatIf`/report mode; run it, read the plan,

then `-Commit`. Creation control and tenant-wide expiration are the two changes most worth a second look.

- **Pilot scope with `Selected`, expand to `All`.** Use `-ManagedGroupTypes Selected` to validate

expiration on a handful of groups before switching to `All`.

- **Announce high-impact changes.** Turning off open creation, enforcing naming, and enabling tenant-wide

expiration are visible to end users — communicate ahead of the commit.

- **Keep the baseline.** Retain the Stage-1 inventory as the "before" record and rollback reference.
- **No secrets in code.** Use interactive modern-auth `Connect-*` (or a managed identity / app registration

with least-privilege consent for scheduled runs) — never hardcode credentials.

- **Verify plan- and cloud-dependent behavior in your tenant.** Entra ID P1 gating, Purview plan coverage,

label-sync cmdlet names, deleted-group restore windows, and Government-cloud portal URLs (`*.office365.us`) all vary — confirm rather than assume.

Reporting & continuous improvement

- **Report on a schedule and trend it.** Run `Get-TeamsGovernanceReport.ps1` monthly; a falling

ownerless/stale/duplicate count is the proof the governance model is working.

- **Close the loop.** Feed report findings (new duplicates, recurring bad names) back into the naming

standard, blocked-words list, and creator training.

Deliverables Summary

Everything included in **Microsoft Teams Governance & Lifecycle Management**, at a glance.

Documentation

- **Solution Overview** (`01-solution-overview.md`) — the governance model (creation control, naming/

classification, ownership, expiration/retention/archival, reporting), reference design with named artifacts, benefits, limitations, and the Assess→Design→Configure→Pilot→Validate→Expand model.

- **Requirements Guide** (`02-requirements.md`) — solution-specific prerequisites (Entra ID P1, Graph +

beta directory-setting cmdlets, creators security group, Purview plan, published container labels, the agreed standard) on top of the shared Teams global requirements — each with Verify / Implement.

- **Implementation Guide** (`03-implementation.md`) — exact ordered steps to set creation control + naming,

classification, ownership, and expiration/retention/archival — pilot-first, `-WhatIf` before commit, with pass/fail gates and a documented rollback. Every artifact named to convention.

- **Testing & Validation** (`05-testing.md`) — 10 procedures, each with an expected PASS/FAIL signal.
- **Troubleshooting** (`06-troubleshooting.md`) — common failure modes and resolutions.
- **Best Practices** (`07-best-practices.md`) — durable governance/lifecycle practices.

- **Executive Summary** (`executive-summary.md`) — the outcome, approach, and why it is low-risk.

PowerShell scripts (5)

All scripts use modern-auth `Connect-*` (no stored credentials), are **read-only or `-WhatIf` by default**, and require an explicit `-Commit` to write.

1. `Get-TeamsGovernanceInventory.ps1` — *Discovery / gap assessment (read-only).* Inventories every

team/M365 Group and flags **ownerless**, **stale**, **duplicate**, and **unclassified** teams; reports whether open creation is still on. Exports the baseline CSV.

1. `Set-TeamsCreationNamingPolicy.ps1` — *Creation-control + naming config helper (`-WhatIf`/`-Commit`).*

Sets `EnableGroupCreation=false` + `GroupCreationAllowedGroupId` (the creators group), applies the prefix/suffix naming requirement + blocked words, and optionally enables container labels. `-Rollback` restores open creation.

1. `Set-TeamsLifecyclePolicy.ps1` — *Expiration / retention / archival config helper (`-WhatIf`/

`-Commit`).* Creates the M365 Groups expiration policy (scope All/Selected), creates a Purview retention policy **only when a compliance-supplied duration is passed**, and archives stale teams read-only via `Set-TeamArchivedState`. Supports `-RemoveExpiration` and un-archive for rollback.

1. `Get-TeamsGovernanceReport.ps1` — *Governance / lifecycle report (read-only).* One schedulable

report combining control posture (creation, naming, classification, expiration) with estate metrics (ownerless/stale/duplicate/unclassified). Exports CSV + HTML for trending.

1. `Test-TeamsGovernanceReadiness.ps1` — *Validation / scope-health helper (read-only).* Emits PASS/FAIL

per governance gate (creation controlled, naming enforced, classification on, expiration active, ownership, stale handled) and exits non-zero on failure so it can gate a rollout.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

Get-TeamsGovernanceInventory.ps1

Get-TeamsGovernanceReport.ps1

Set-TeamsCreationNamingPolicy.ps1

Set-TeamsLifecyclePolicy.ps1

Test-TeamsGovernanceReadiness.ps1